TL3 Hiperparámetros

Índice

1. Introducción
2. SGD
3. Validación
4. Validación cruzada
5. Exploración de hiperparámetros
6. Ejercicio

Cita

Juan Ciscar, Alfons. "Material docente de la asignatura Aprendizaje Automático (APA)". Grado de Inteligencia Artificial (GIAR), Escuela Técnica Superior de Ingeniería Informática (ETSINF), Dep. de Sistemas Informáticos y Computación (DSIC), Universitat Politècnica de València (UPV), primer semestre del curso 2026/27.

Copyright © 2026 Alfons Juan Ciscar

1 Introducción

Objetivo: introducir herramientas estándar para realizar experimentos de ajuste de hiperparámetros

SGD: usamos LogisticRegression en lugar de SGDClassifier con `loss=log_loss`

Validación: mediante la popular train_test_split

Validación cruzada: en K bloques o con divisiones aleatorias

Exploración de hiperparámetros: mediante exploración exhaustiva o aleatoria en una rejilla

Copyright © 2026 Alfons Juan Ciscar

2 SGD

SGD en scikit-learn: SGDClassifier con `loss=log_loss` es regresión logística ajustada con SGD

LogisticRegression: es regresión logística ajustada con otros solvers; por omisión, `solver=lbfgs` por su robustez

MNIST: aunque cabe esperar resultados similares, usamos LogisticRegression en lugar de SGDClassifier

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("ylecun/mnist").with_format("numpy")
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
K = 400; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train); X_test = pca.transform(X_test)
print(X_train.shape, y_train.shape, X_test.shape, y_test.shape)
```

```
(60000, 400) (60000,) (10000, 400) (10000,)
```

In [ ]:
```
from sklearn.linear_model import LogisticRegression
acc = LogisticRegression(C=0.1, max_iter=200).fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión LogisticRegression: {acc:.1%}")
```

```
Precisión LogisticRegression: 92.6%
```

In [ ]:
```
from sklearn.linear_model import SGDClassifier
acc = SGDClassifier(loss='log_loss').fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión SGDClassifier: {acc:.1%}")
```

```
Precisión SGDClassifier: 91.7%
```

Ejercicio: compara LogisticRegression y SGDClassifier en Fashion-MNIST con $K = 600$ y CIFAR-10 con $K = 1000$

Copyright © 2026 Alfons Juan Ciscar

Solución:

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression, SGDClassifier
ds = datasets.load_dataset("zalando-datasets/fashion_mnist").with_format("numpy")
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
K = 600; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train); X_test = pca.transform(X_test)
acc_lr = LogisticRegression(C=0.1, max_iter=200).fit(X_train, y_train).score(X_test, y_test)
acc_sgd = SGDClassifier(loss='log_loss').fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión Fashion-MNIST:  LogisticRegression: {acc_lr:.1%}, SGDClassifier: {acc_sgd:.1%}")
```

```
Precisión Fashion-MNIST:  LogisticRegression: 84.5%, SGDClassifier: 83.9%
```

In [ ]:
```
ds = datasets.load_dataset("uoft-cs/cifar10").with_format("numpy")
X_train = ds['train'][:]['img'].astype(np.float32).reshape(-1, 32*32*3) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['img'].astype(np.float32).reshape(-1, 32*32*3) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
K = 1000; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train); X_test = pca.transform(X_test)
acc_lr = LogisticRegression(C=0.1, max_iter=200).fit(X_train, y_train).score(X_test, y_test)
acc_sgd = SGDClassifier(loss='log_loss').fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión CIFAR-10:  LogisticRegression: {acc_lr:.1%}, SGDClassifier: {acc_sgd:.1%}")
```

```
Precisión CIFAR-10:  LogisticRegression: 40.5%, SGDClassifier: 37.4%
```

Copyright © 2026 Alfons Juan Ciscar

3 Validación

Validación: método popular y sencillo para ajustar un vector de hiperparámetros $\boldsymbol{\lambda}$

- Dividimos los datos en un conjunto de **entrenamiento** $\mathcal{D}_{\text{train}}$ y otro de **validación o desarrollo** $\mathcal{D}_{\text{valid}}$
- Cada valor de $\boldsymbol{\lambda}$ se evalúa con $\mathcal{D}_{\text{valid}}$ tras ajustar el modelo con $\mathcal{D}_{\text{train}}$
- Elegimos el $\boldsymbol{\lambda}$ que proporciona mejor rendimiento en $\mathcal{D}_{\text{valid}}$

El test no se usa: $\mathcal{D}_{\text{train}}$ y $\mathcal{D}_{\text{valid}}$ se obtienen dividiendo los datos de entrenamiento, nunca los de test

Si el test se usa muchas veces: es más validación que test y el mejor resultado obtenido es optimista

train_test_split: divide arrays o matrices en subconjuntos aleatorios de train y test

- `test_size=None`: fracción de datos a incluir en el test; `None` equivale a 0.25 (25%)
- `train_size=None`: fracción de datos a incluir en el train; `None` equivale a escoger el complemento del test
- `random_state=None`: semilla para controlar el barajado previo a la división
- `shuffle=True`: barajar o no los datos antes de dividirlos
- `stratify=None`: etiquetado de los datos para producir una división estratificada (por clases)

Ejemplo: división aleatoria de 10 datos de 2 clases en un train con 7 y un test con 3

In [ ]:
```
import numpy as np; from sklearn.model_selection import train_test_split
X = np.arange(10); y = np.hstack((np.repeat(0, 5), np.repeat(1, 5)))
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=23)
np.vstack((X_train, y_train)), np.vstack((X_test, y_test))
```

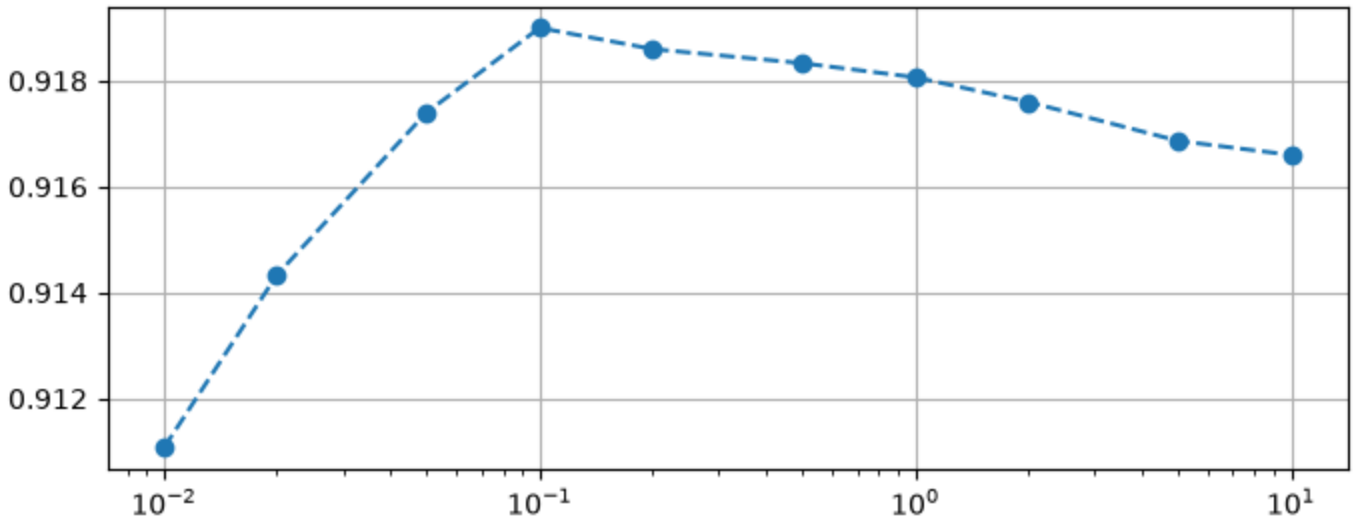
Out[ ]:
```
(array([[9, 4, 7, 1, 0, 6, 3],
        [1, 0, 1, 0, 0, 1, 0]]),
 array([[5, 8, 2],
        [1, 1, 0]]))
```

MNIST: ajuste del hiperparámetro `C` de `LogisticRegression`; positivo, cuanto mayor C, mayor desregularización

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("ylecun/mnist").with_format("numpy")
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
K = 400; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train); X_test = pca.transform(X_test)
```

In [ ]:
```
from sklearn.model_selection import train_test_split; from sklearn.linear_model import LogisticRegression
X_tr2, X_valid, y_tr2, y_valid = train_test_split(X_train, y_train, test_size=0.25, random_state=23)
Cs = np.array([.01, .02, .05, .1, .2, .5, 1, 2, 5, 10]); acc = np.zeros_like(Cs)
for i, C in enumerate(Cs): acc[i] = LogisticRegression(C=C, max_iter=200).fit(X_tr2, y_tr2).score(X_valid, y_valid)
```

In [ ]:
```
import matplotlib.pyplot as plt; plt.figure(figsize=(8, 3)); plt.grid(); plt.semilogx(Cs, acc, 'o--');
```



Evaluación: evaluación en test tras entrenar con todo el train y el mejor C hallado

In [ ]:
```
C_star=.1; acc = LogisticRegression(C=C_star, max_iter=200).fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión LogisticRegression con C = {C_star}: {acc:.1%}")
```

```
Precisión LogisticRegression con C = 0.1: 92.6%
```

Copyright © 2026 Alfons Juan Ciscar

4 Validación cruzada

Kfold cross validation (KCV): versión refinada de validación que aprovecha todos los datos para validar

- Dividimos los datos en K bloques, $\mathcal{D}_1, \ldots, \mathcal{D}_K$
- Cada valor de $\boldsymbol{\lambda}$ se evalúa con cada bloque $\mathcal{D}_k$ tras ajustar el modelo con el resto de bloques, $\mathcal{D}_{-k}$
- Elegimos el $\boldsymbol{\lambda}$ que proporciona mejor rendimiento promedio

KFold: validador cruzado en K bloques

- `n_splits=5`: K
- `shuffle=False`: barajar o no los datos antes de particionarlos en K bloques
- `random_state=None`: semilla para controlar el barajado previo a la partición; solo con `shuffle=True`

Ejemplo: particiones de 10 datos de 2 clases con 5 bloques

In [ ]:
```
import numpy as np; from sklearn.model_selection import KFold
X = np.arange(10); kf = KFold(n_splits=5, shuffle=True, random_state=23)
for i, (X_train_ind, X_valid_ind) in enumerate(kf.split(X)):
    print(f"Split {i}: {X_train_ind} {X_valid_ind}")
```

```
Split 0: [0 1 2 3 4 6 7 9] [5 8]
Split 1: [0 1 3 4 5 6 7 8] [2 9]
Split 2: [0 1 2 3 5 6 8 9] [4 7]
Split 3: [2 3 4 5 6 7 8 9] [0 1]
Split 4: [0 1 2 4 5 7 8 9] [3 6]
```

cross_val_score: función auxiliar para simplificar la evaluación del rendimiento con validación cruzada

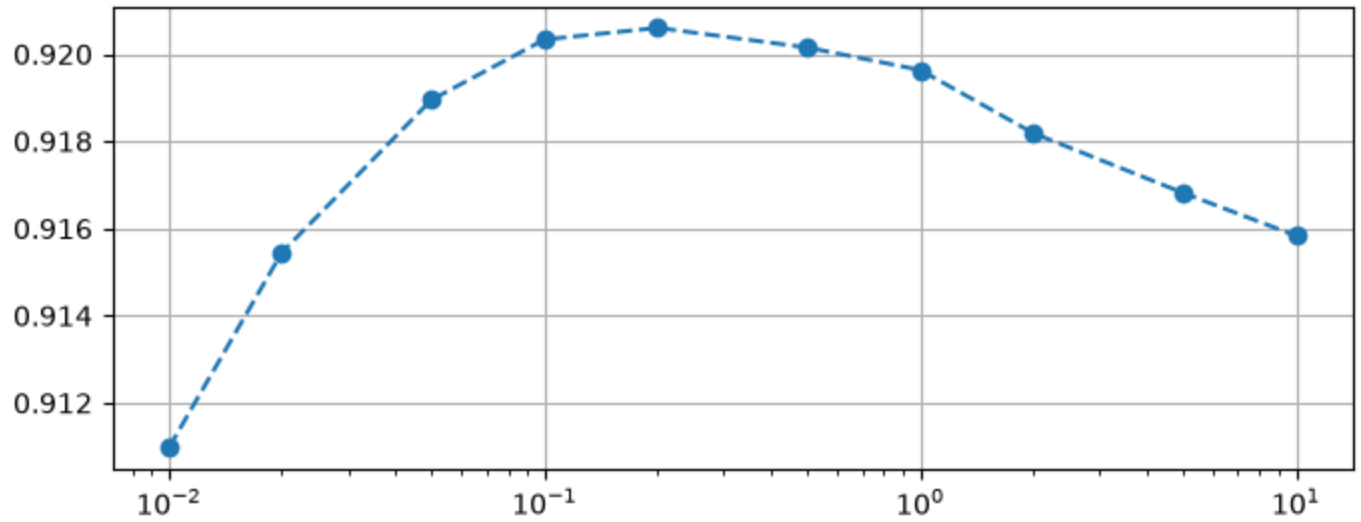
Copyright © 2026 Alfons Juan Ciscar

MNIST: ajuste del hiperparámetro `C` de `LogisticRegression`; positivo, cuanto mayor C, mayor desregularización

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("ylecun/mnist").with_format("numpy")
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
K = 400; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train); X_test = pca.transform(X_test)
```

In [ ]:
```
import matplotlib.pyplot as plt; from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression; import time; start = time.time()
Cs = np.array([.01, .02, .05, .1, .2, .5, 1, 2, 5, 10]); acc = np.zeros_like(Cs)
for i, C in enumerate(Cs):
    acc[i] = cross_val_score(LogisticRegression(C=C, max_iter=200), X_train, y_train, cv=4).mean()
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
plt.figure(figsize=(8, 3)); plt.grid(); plt.semilogx(Cs, acc, 'o--');
```

```
Tiempo (hh:mm:ss): 00:07:42
```



In [ ]:
```
C_star=.2; acc = LogisticRegression(C=C_star, max_iter=200).fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión LogisticRegression con C = {C_star}: {acc:.1%}")
```

```
Precisión LogisticRegression con C = 0.2: 92.6%
```

Validación cruzada con divisiones aleatorias: estrategia popular, más flexible que KCV

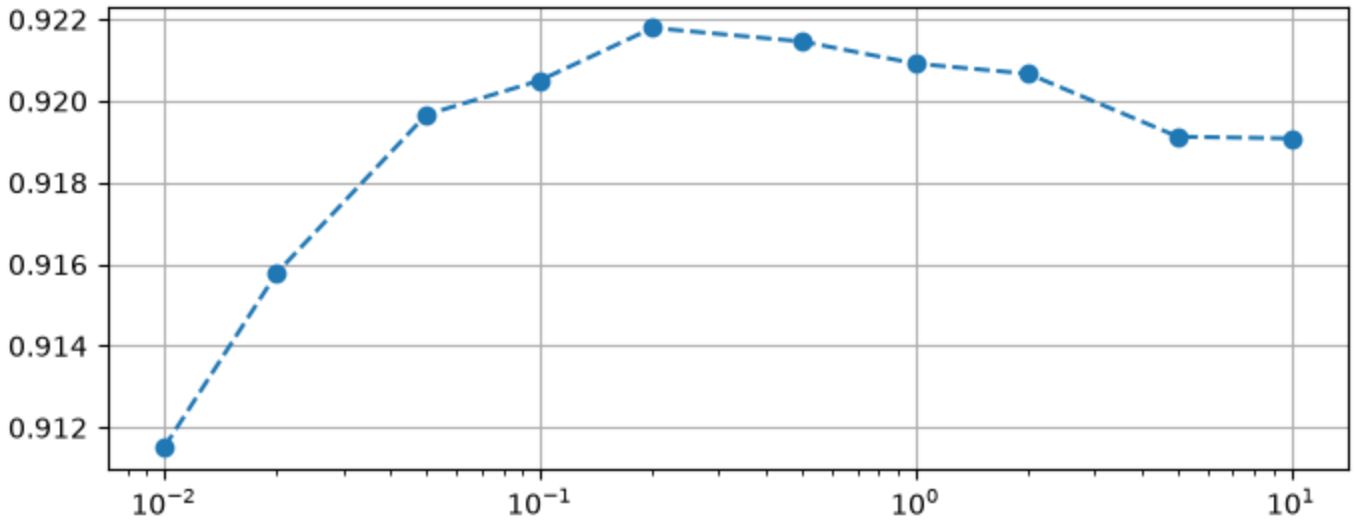
ShuffleSplit: validador cruzado mediante divisiones aleatorias

- `n_splits=10`: número de divisiones aleatorias
- `test_size=None`: fracción de test; `None` para escoger el complemento del train; `0.1` si `train_size=None`
- `train_size=None`: fracción de train; `None` para escoger el complemento del test
- `random_state=None`: semilla para controlar las divisiones aleatorias

MNIST: ajuste del hiperparámetro `C` de `LogisticRegression`

In [ ]:
```
import matplotlib.pyplot as plt; from sklearn.model_selection import ShuffleSplit, cross_val_score
from sklearn.linear_model import LogisticRegression; import time; start = time.time()
Cs = np.array([.01, .02, .05, .1, .2, .5, 1, 2, 5, 10]); acc = np.zeros_like(Cs)
splitter = ShuffleSplit(n_splits=4, test_size=0.1, random_state=23)
for i, C in enumerate(Cs):
    acc[i] = cross_val_score(LogisticRegression(C=C, max_iter=200), X_train, y_train, cv=splitter).mean()
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
plt.figure(figsize=(8, 3)); plt.grid(); plt.semilogx(Cs, acc, 'o--');
```

```
Tiempo (hh:mm:ss): 00:07:35
```



5 Exploración de hiperparámetros

GridSearchCV: búsqueda exhaustiva en una rejilla de valores

RandomizedSearchCV: búsqueda en una rejilla de valores de `n_iter=10` valores escogidos aletoriamente

MNIST: exploración exhaustiva de 30 valores vs aleatoria de 10

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("ylecun/mnist").with_format("numpy")
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
K = 400; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train); X_test = pca.transform(X_test)
```

In [ ]:
```
import warnings; warnings.filterwarnings('ignore'); from sklearn.model_selection import ShuffleSplit, GridSearchCV
from sklearn.linear_model import LogisticRegression; import time; start = time.time()
clf = LogisticRegression(); G = {"C": [.01, .02, .05, .1, .2, .5, 1, 2, 5, 10], "max_iter": [100, 200, 300]}
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
GS = GridSearchCV(clf, G, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=8)
acc = GS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.1%} con {GS.best_params_}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

```
Precisión: 92.6% con {'C': 0.1, 'max_iter': 100}
Tiempo (hh:mm:ss): 00:00:35
```

In [ ]:
```
from sklearn.model_selection import RandomizedSearchCV; import time; start = time.time()
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
GS = RandomizedSearchCV(clf, G, n_iter=10, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=8)
acc = GS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.1%} con {GS.best_params_}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

```
Precisión: 92.6% con {'max_iter': 200, 'C': 0.1}
Tiempo (hh:mm:ss): 00:00:15
```

Copyright © 2026 Alfons Juan Ciscar

4 Ejercicio

Ejercicio: reproduce el experimento de exploración de hiperparámetros con MNIST en Fashion-MNIST con $K = 600$ y CIFAR-10 con $K = 1000$

Fashion-MNIST

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("zalando-datasets/fashion_mnist").with_format("numpy")
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
K = 600; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train); X_test = pca.transform(X_test)
```

In [ ]:
```
import warnings; warnings.filterwarnings('ignore'); from sklearn.model_selection import ShuffleSplit, GridSearchCV
from sklearn.linear_model import LogisticRegression; import time; start = time.time()
clf = LogisticRegression(); G = {"C": [.01, .02, .05, .1, .2, .5, 1, 2, 5, 10], "max_iter": [100, 200, 300]}
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
GS = GridSearchCV(clf, G, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=8)
acc = GS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.1%} con {GS.best_params_}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

```
Precisión: 84.5% con {'C': 0.05, 'max_iter': 300}
Tiempo (hh:mm:ss): 00:01:44
```

In [ ]:
```
from sklearn.model_selection import RandomizedSearchCV; import time; start = time.time()
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
GS = RandomizedSearchCV(clf, G, n_iter=10, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=8)
acc = GS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.1%} con {GS.best_params_}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

```
Precisión: 84.4% con {'max_iter': 100, 'C': 0.2}
Tiempo (hh:mm:ss): 00:00:43
```

Copyright © 2026 Alfons Juan Ciscar

CIFAR-10

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("uoft-cs/cifar10").with_format("numpy")
X_train = ds['train'][:]['img'].astype(np.float32).reshape(-1, 32*32*3) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
X_test = ds['test'][:]['img'].astype(np.float32).reshape(-1, 32*32*3) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
labels = ("airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck")
K = 1000; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train); X_test = pca.transform(X_test)
```

In [ ]:
```
import warnings; warnings.filterwarnings('ignore'); from sklearn.model_selection import ShuffleSplit, GridSearchCV
from sklearn.linear_model import LogisticRegression; import time; start = time.time()
clf = LogisticRegression(); G = {"C": [.01, .02, .05, .1, .2, .5, 1, 2, 5, 10], "max_iter": [100, 200, 300]}
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
GS = GridSearchCV(clf, G, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=8)
acc = GS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.1%} con {GS.best_params_}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

```
Precisión: 41.8% con {'C': 0.01, 'max_iter': 200}
Tiempo (hh:mm:ss): 00:03:11
```

In [ ]:
```
from sklearn.model_selection import RandomizedSearchCV; import time; start = time.time()
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
GS = RandomizedSearchCV(clf, G, n_iter=10, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=8)
acc = GS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.1%} con {GS.best_params_}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

```
Precisión: 41.8% con {'max_iter': 200, 'C': 0.01}
Tiempo (hh:mm:ss): 00:02:14
```

Copyright © 2026 Alfons Juan Ciscar